

实验题目： Tasklet 和工作队列

一、 实验内容

按实验内容编写驱动程序及测试程序

编译驱动程序

在 Ubuntu 上加载驱动程序并进行测试

分析对比 Tasklet 和工作队列的差异

写一个简单的字符设备驱动程序，要求：

定义一个 Tasklet 和一个工作队列，实现打印输出；

定义两个定时器，定时器周期分别为 T1 和 T2；

T1 周期到时调度 Tasklet，T2 周期到时调度工作队列；

在加载驱动模块时注册 Tasklet 和工作队列

在卸载驱动模块时销毁 Tasklet 和工作队列

注意事项：

在编写驱动程序及测试程序时，在关键位置增加打印信息，使得测试结果打印输出可以显示驱动程序加载卸载以及应用程序调用驱动程序的过程。

驱动程序打印调用 printk 函数

在 ubuntu 虚拟机上完成程序验证

二、 实验原理（代码以及注释）

demo_drv3.c:

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/time.h>
#include <linux/timer.h>
#include <linux/workqueue.h>
#include <linux/atomic.h>
#include <linux/fs.h>
#include <linux/interrupt.h>

/* 定时器数据结构 */
struct timer_data {
    struct timer_list timer;    // 内核定时器
    unsigned long prev_jiffies; // 上次触发时间戳
    unsigned int loops;        // 触发次数统计
};

/* 声明两个定时器实例 */
struct timer_data test_data1; // 定时器 T1（2 秒周期）
struct timer_data test_data2; // 定时器 T2（3 秒周期）

/* Tasklet 相关声明 */
static struct tasklet_struct my_tasklet; // Tasklet 结构体

/* 工作队列相关声明 */
static struct workqueue_struct *my_workqueue; // 工作队列指针
static void do_work(struct work_struct *work); // 工作处理函数原型
static DECLARE_WORK(test_work, do_work); // 静态声明工作项

/* 原子变量记录工作队列执行次数 */
```

```

static atomic_t wq_run_times = ATOMIC_INIT(0);           // 初始化为 0
/* 普通变量记录入队失败次数 */
static unsigned int failed_cnt = 0;

/***** Tasklet 处理函数 *****/
static void tasklet_handler(unsigned long data) {
    /* 此处执行实际任务，这里仅打印信息 */
    printk(KERN_INFO "Tasklet triggered by Timer T1!\n");
}

/***** 工作队列处理函数 *****/
static void do_work(struct work_struct *work) {
    /* 原子递增执行次数 */
    atomic_inc(&wq_run_times);

    /* 打印执行信息 */
    printk(KERN_INFO "Workqueue triggered by Timer T2! Execution count: %d\n",
        atomic_read(&wq_run_times));
}

/***** 定时器 T1 回调函数 *****/
void test_timer_fn1(unsigned long arg) {
    struct timer_data *data = (struct timer_data *)arg;
    unsigned long j = jiffies; // 获取当前时间戳

    /* 重新设置定时器（2 秒后再次触发） */
    data->timer.expires = j + 2 * HZ;
    data->prev_jiffies = j;
    add_timer(&data->timer);    // 重新激活定时器

    /* 调度 Tasklet */
    tasklet_schedule(&my_tasklet);
    printk(KERN_INFO "Timer T1 expired! Scheduling Tasklet...\n");
}

/***** 定时器 T2 回调函数 *****/
void test_timer_fn2(unsigned long arg) {
    struct timer_data *data = (struct timer_data *)arg;
    unsigned long j = jiffies;

    /* 重新设置定时器（3 秒后再次触发） */
    data->timer.expires = j + 3 * HZ;
    data->prev_jiffies = j;
    add_timer(&data->timer);

    /* 将工作项加入工作队列 */
    if (!queue_work(my_workqueue, &test_work)) {

```

```

        /* 入队失败处理 */
        printk(KERN_WARNING "Timer T2 expired but workqueue is busy!\n");
        failed_cnt++;
    } else {
        printk(KERN_INFO "Timer T2 expired! Work queued successfully.\n");
    }
}

/***** 模块初始化 *****/
static int __init demo_init(void) {
    printk(KERN_INFO "Driver loading...\n");
    printk(KERN_INFO "Initial jiffies: %lu\n", jiffies);

    /* 初始化原子变量 */
    atomic_set(&wq_run_times, 0);

    /***** 初始化定时器 T1 (2 秒周期) *****/
    init_timer(&test_data1.timer);
    test_data1.timer.function = test_timer_fn1; // 设置回调函数
    test_data1.timer.data = (unsigned long)&test_data1; // 传递数据指针
    test_data1.timer.expires = jiffies + 2 * HZ; // 首次到期时间
    test_data1.prev_jiffies = jiffies;
    test_data1.loops = 0;
    add_timer(&test_data1.timer); // 启动定时器
    printk(KERN_INFO "Timer T1 initialized (2s interval)\n");

    /***** 初始化定时器 T2 (3 秒周期) *****/
    init_timer(&test_data2.timer);
    test_data2.timer.function = test_timer_fn2;
    test_data2.timer.data = (unsigned long)&test_data2;
    test_data2.timer.expires = jiffies + 3 * HZ;
    test_data2.prev_jiffies = jiffies;
    test_data2.loops = 0;
    add_timer(&test_data2.timer);
    printk(KERN_INFO "Timer T2 initialized (3s interval)\n");

    /***** 初始化 Tasklet *****/
    tasklet_init(&my_tasklet, // 结构体指针
                tasklet_handler, // 处理函数
                0); // 无传递数据
    printk(KERN_INFO "Tasklet initialized\n");

    /***** 创建工作队列 *****/
    my_workqueue = create_singlethread_workqueue("my_wq"); // 单线程工作队列
    if (!my_workqueue) {
        printk(KERN_ERR "Failed to create workqueue!\n");
        return -ENOMEM;
    }
}

```

```

    }
    printk(KERN_INFO "Workqueue created\n");

    printk(KERN_INFO "Driver loaded successfully\n");
    return 0;
}

/***** 模块卸载 *****/
static void __exit demo_exit(void) {
    /* 删除定时器 */
    del_timer_sync(&test_data1.timer);
    del_timer_sync(&test_data2.timer);

    /* 销毁 Tasklet */
    tasklet_kill(&my_tasklet);

    /* 销毁工作队列 */
    if (my_workqueue) {
        flush_workqueue(my_workqueue);    // 确保所有工作完成
        destroy_workqueue(my_workqueue);
    }

    /* 打印统计信息 */
    printk(KERN_INFO "Workqueue executed %d times, failed enqueues: %u\n",
           atomic_read(&wq_run_times), failed_cnt);

    printk(KERN_INFO "Driver unloaded\n");
}

/* 模块元信息 */
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("A driver with timer, tasklet and workqueue");

/* 注册初始化/退出函数 */
module_init(demo_init);
module_exit(demo_exit);

```

三、 实验结果及其分析

编译成功后，加载驱动程序，查看日志打印信息：

可见驱动成功初始化，定时器，tasklet，工作队列均成功初始化：

```

[ 2377.655370] Driver loading...
[ 2377.655374] Initial jiffies: 4295488157
[ 2377.655376] Timer T1 initialized (2s interval)
[ 2377.655378] Timer T2 initialized (3s interval)
[ 2377.655379] Tasklet initialized
[ 2377.655622] Workqueue created
[ 2377.655623] Driver loaded successfully
[ 2379.656617] Timer T1 expired! Scheduling Tasklet...
[ 2379.656623] Tasklet triggered by Timer T1!
[ 2380.653302] Timer T2 expired! Work queued successfully.
[ 2380.653409] Workqueue triggered by Timer T2! Execution count: 1
[ 2381.653721] Timer T1 expired! Scheduling Tasklet...
[ 2381.653727] Tasklet triggered by Timer T1!
[ 2383.664747] Timer T2 expired! Work queued successfully.
[ 2383.664749] Timer T1 expired! Scheduling Tasklet...
[ 2383.664750] Tasklet triggered by Timer T1!
[ 2383.664755] Workqueue triggered by Timer T2! Execution count: 2
[ 2385.676680] Timer T1 expired! Scheduling Tasklet...
[ 2385.676686] Tasklet triggered by Timer T1!
[ 2386.660780] Timer T2 expired! Work queued successfully.
[ 2386.660798] Workqueue triggered by Timer T2! Execution count: 3
[ 2387.676363] Timer T1 expired! Scheduling Tasklet...
[ 2387.676368] Tasklet triggered by Timer T1!

```

查看实时变动，可见每 2 秒调度 tasklet (T1)，每 3 秒调度工作队列 (T2)：

```

Apr 19 11:21:49 ubuntu kernel: [ 2445.766134] Tasklet triggered by Timer T1!
Apr 19 11:21:50 ubuntu kernel: [ 2446.734425] Timer T2 expired! Work queued successfully.
Apr 19 11:21:50 ubuntu kernel: [ 2446.734434] Workqueue triggered by Timer T2! Execution count: 23
Apr 19 11:21:51 ubuntu kernel: [ 2447.769196] Timer T1 expired! Scheduling Tasklet...
Apr 19 11:21:51 ubuntu kernel: [ 2447.769209] Tasklet triggered by Timer T1!
Apr 19 11:21:53 ubuntu kernel: [ 2449.734594] Timer T2 expired! Work queued successfully.
Apr 19 11:21:53 ubuntu kernel: [ 2449.734781] Workqueue triggered by Timer T2! Execution count: 24
Apr 19 11:21:53 ubuntu kernel: [ 2449.766838] Timer T1 expired! Scheduling Tasklet...
Apr 19 11:21:53 ubuntu kernel: [ 2449.766844] Tasklet triggered by Timer T1!
Apr 19 11:21:55 ubuntu kernel: [ 2451.766470] Timer T1 expired! Scheduling Tasklet...
Apr 19 11:21:55 ubuntu kernel: [ 2451.766475] Tasklet triggered by Timer T1!
Apr 19 11:21:56 ubuntu kernel: [ 2452.736842] Timer T2 expired! Work queued successfully.
Apr 19 11:21:56 ubuntu kernel: [ 2452.736989] Workqueue triggered by Timer T2! Execution count: 25
Apr 19 11:21:57 ubuntu kernel: [ 2453.764946] Timer T1 expired! Scheduling Tasklet...
Apr 19 11:21:57 ubuntu kernel: [ 2453.764956] Tasklet triggered by Timer T1!
Apr 19 11:21:59 ubuntu kernel: [ 2455.735897] Timer T2 expired! Work queued successfully.
Apr 19 11:21:59 ubuntu kernel: [ 2455.735916] Workqueue triggered by Timer T2! Execution count: 26
Apr 19 11:21:59 ubuntu kernel: [ 2455.766418] Timer T1 expired! Scheduling Tasklet...
Apr 19 11:21:59 ubuntu kernel: [ 2455.766423] Tasklet triggered by Timer T1!

```

最后卸载驱动程序，可见定时器，tasklet，工作队列均被销毁：

```
2471.775336] Timer T1 expired! Scheduling Tasklet...
2471.775346] Tasklet triggered by Timer T1!
2473.750656] Timer T2 expired! Work queued successfully.
2473.750678] Workqueue triggered by Timer T2! Execution count: 32
2473.776760] Timer T1 expired! Scheduling Tasklet...
2473.776770] Tasklet triggered by Timer T1!
2475.775236] Timer T1 expired! Scheduling Tasklet...
2475.775242] Tasklet triggered by Timer T1!
2476.749226] Timer T2 expired! Work queued successfully.
2476.749246] Workqueue triggered by Timer T2! Execution count: 33
2477.776249] Timer T1 expired! Scheduling Tasklet...
2477.776255] Tasklet triggered by Timer T1!
2479.747997] Timer T2 expired! Work queued successfully.
2479.749862] Workqueue triggered by Timer T2! Execution count: 34
2479.772736] Timer T1 expired! Scheduling Tasklet...
2479.772741] Tasklet triggered by Timer T1!
2481.775255] Timer T1 expired! Scheduling Tasklet...
2481.775265] Tasklet triggered by Timer T1!
2482.753685] Timer T2 expired! Work queued successfully.
2482.753719] Workqueue triggered by Timer T2! Execution count: 35
2483.777814] Timer T1 expired! Scheduling Tasklet...
2483.777822] Tasklet triggered by Timer T1!
2485.760603] Timer T2 expired! Work queued successfully.
2485.761981] Workqueue triggered by Timer T2! Execution count: 36
2485.790862] Timer T1 expired! Scheduling Tasklet...
2485.790870] Tasklet triggered by Timer T1!
2487.709316] Workqueue executed 36 times, failed enqueues: 0
2487.709319] Driver unloaded
```